

Objects, While & For Loops

OK, right now our program can look at a list of products to buy, **choose to buy one of them**. What if we want to buy more than one thing? We'd need a SHOPPING cart. That is an object, *an object* literally just describes a thing;

- So for objects, we view a dictionary as a way to describe something.

Let's make a "Person" Object, again a dictionary.

```
person = {"name": "Alice", "age": 30, "city": "New York"}
```

Instead of treating this as a list of products, *it's the characteristics that explain a person*.

- **Just** like a dictionary, the left side is unique; you can only have one *name*, one *age*, one *city*. Then there's the value.

Let's expand our program to instead of simply grabbing inputs; **Let's have it pull from the person object**.

- Have the program grab the **Balance**, out of the person object (*just like a dictionary, it'd just grab it via `person["balance"]`*).
- And give the PERSON a list that fills up with the products as the person picks up products; i.e. the person's SHOPPING cart.
- Let us also add a "Discount" in case the person is an employee, or a veteran.

```
person = {"name": "Bob", "balance": 100, "cart": [], "discount": 0.1}
```

We still have our dictionary of products, *which we're still treating like a normal list*; Which the user will use to add to their shopping cart.

```
products = {"strawberry": 0.8, "banana": 0.5, "apple": 0.3}
```

Now, let's think; **AS A CUSTOMER** I need to be able to checkout, for now let's just give the customer a pre made shopping cart.

- By that I mean we **ADDED** *products* from our *products* dictionary.

```
person = {"name": "Bob", "balance": 100, "discount": 0.1, "cart": [{  
    "strawberry": 2 }, { "banana": 3 }]}
```

To pay, we need to **go through EACH product** we have in our shopping cart. And add up the costs.

- When you know you need to “GO THROUGH EACH” you need to use a FOR LOOP.

Our, price is calculated in this total here.

```
total = 0
```

```
for item in person["cart"]:
```

What, this does is SAVE EACH value in the list as “item”. ITEM, acts as the next item in the “cart” list.

- So the first time it loops item is equal to {"strawberry": 2}
- The next time it loops, item is equal to {"banana": 3}

```
    for price in item.values():
```

```
        total += price
```

THIS, is inside the one above. So it does below, individually for each item (i.e. each product in your cart)

- When grabbing out values from a dictionary, you can do `dictionary.values()`, this creates a list all of the values from the dictionary.
- PRICE, is equal to the actual value from the list, and changes each loop. *There's only one value in each product to loop through.*
 - So it adds up each price to a total.

```
person = {"name": "Bob", "balance": 100, "discount": 0.1, "cart": [{
```

```
"strawberry": 2 }, { "banana": 3 }]]
```

```
products = {"strawberry": 2, "banana": 3, "apple": 0.3}
```

```
total = 0
```

```
for item in person["cart"]:
```

```
    for price in item.values():
```

```
        total += price
```

```
print(total)
```

So to recap, FOR LOOPS; go through a list (i.e. `person["cart"]`, or `item.values()`), and you save a variable (i.e. `item` or `price`) which is each actual values inside that list. I.e. the first value in the list, then the 2nd until the list is done.

Let's do another for loop, this time more simple. For loops for dictionaries are weirder than normal lists.

- The customer will want to be able to SEE what's in it's cart, SO we can simply just use a for loop like below to print the entire list.

```
for item in person["cart"]:
```

```
    print(item)
```

Again, "item" is just some random variable that is equal to the next value in the list, that list being person["cart"].

- person["cart"] is a list with 2 values,
 - So in the **first LOOP**, item will equal `{ "strawberry": 2 }`
 - In the second item will then become `{ "banana": 3 }`
-

That's the GIST of **for loops**; but right now we just have code, not really a shopping program.

To organize things, in coding we normally tend to take our chunks of code like our *print shopping cart OR getting the shopping cart total*, and create them into something called a function of our own, and USE THAT FUNCTION, in the place that the user normally interacts. This will make more sense in the next part.